

Java collections

Contents

1. [What is the Collections Framework, and why was it introduced?](#)
2. [Collection framework diagram](#)

1. What is the Collections Framework, and why was it introduced?

The Java Collections Framework is the standard library in `java.util` for storing and processing **groups of objects**. It has three parts, and naming all three is the complete answer:

Component	What it gives you
Interfaces	The contracts: <code>List</code> , <code>Set</code> , <code>Queue</code> , <code>Map</code> define what operations a collection supports
Implementations	The concrete classes behind them: <code>ArrayList</code> , <code>HashSet</code> , <code>HashMap</code> , and the rest
Algorithms	Ready-made operations in the <code>Collections</code> utility class: <code>sort</code> , <code>binarySearch</code> , <code>shuffle</code> , <code>max</code> , <code>min</code>

Why it was introduced: before Java 1.2, the language offered `Vector`, `Stack`, `Hashtable`, and plain arrays. Since each had its own unrelated API, nothing interoperated and every team rewrote the same data-structure logic.

The framework replaced that with one unified architecture: code against the interface, and any implementation plugs in. So we get tested, efficient structures for free, algorithms that work across all of them, and the freedom to swap `ArrayList` for `LinkedList` without touching the calling code.

Unified architecture in practice: the core operations (`add`, `remove`, `contains`, `size`, iteration) behave the same way on every collection, so declaring the variable as the interface makes the implementation a one-line decision:

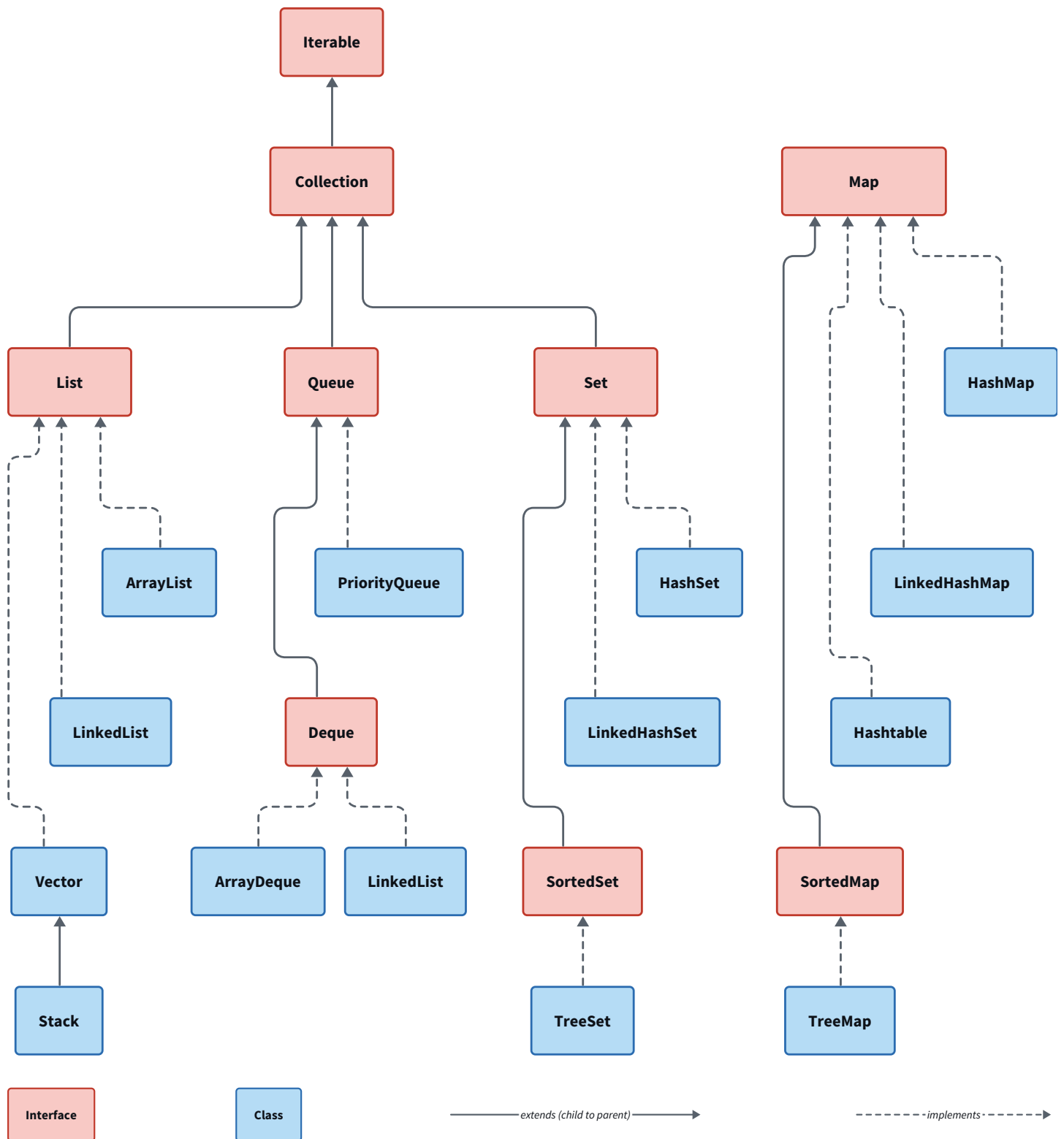
```
// code against the interface
List<String> names = new ArrayList<>();

// later, only the construction line changes;
// every caller stays untouched
List<String> names = new LinkedList<>();
```

The implementations are the classic data structures, ready-made:

Data structure	Framework implementation
Dynamic array	<code>ArrayList</code>
Doubly-linked list	<code>LinkedList</code>
Stack	<code>ArrayDeque</code> (the legacy <code>Stack</code> class is obsolete)
Queue	<code>ArrayDeque</code> ; <code>PriorityQueue</code> for priority order
Hash table	<code>HashMap</code> , <code>HashSet</code>
Hash table with order	<code>LinkedHashMap</code> , <code>LinkedHashSet</code>
Balanced (red-black) tree	<code>TreeMap</code> , <code>TreeSet</code>

2. Collection framework diagram



Legacy interface

Enumeration

`Enumeration` is the legacy interface for traversing a collection, with just two methods: `hasMoreElements()` and `nextElement()`. The older classes such as `Vector` and `Hashtable` still use it.

In modern Java, `Iterator` is preferred, since it also supports removing elements during iteration.

Legacy classes

Vector

`Vector` is a synchronised, dynamically resizable array. Since every method takes a lock, it runs slower than `ArrayList` in single-threaded code, so `ArrayList` is the preferred choice unless thread safety is genuinely required.

Stack

`Stack` extends `Vector` and provides **Last-In, First-Out (LIFO)** operations: `push()`, `pop()`, and `peek()`. As it inherits the legacy synchronisation of `Vector`, `ArrayDeque` is the recommended replacement.

Dictionary

`Dictionary` is an abstract class that stores key-value pairs. It is deprecated in practice, as the `Map` interface replaced it with a more flexible and complete API.

Hashtable

`Hashtable` is a synchronised key-value map that extends `Dictionary`, and it does not allow `null` keys or values. In most cases `HashMap` is used instead, and for thread-safe applications `ConcurrentHashMap` is the preferred modern alternative.

Properties

`Properties` is a subclass of `Hashtable` that stores configuration data as **String key-value pairs**. It is commonly used to read and write `.properties` configuration files.